

Détection de QR Codes malveillants

QR Code Security Analyzer

Projet de fin d'études — Cybersécurité

*Traitement d'image, détection d'altération physique et analyse de réputation
par intelligence des menaces (VirusTotal API v3)*

Étude technique · Architecture en 3 étapes · Système de scoring de risque · Intégration API ·
Rapport PDF

Application Python — OpenCV · Pyzbar · CustomTkinter · ReportLab · VirusTotal API v3

**Réalisé par : Imad Hamdani - Laabadla Yassine - Mohammed Ait Addi - Messaoudi
Youness**

Approche : Security by Design — validation physique avant toute lecture numérique

Table des matières

1	Contexte général, problématique et étude du besoin	3
1.1	Introduction	3
1.2	Contexte général du projet	3
1.3	Problématique métier et technique	3
1.4	Objectifs du projet	3
1.5	Analyse de la menace : le Qishing	3
1.5.1	a. Définition et mécanisme d'attaque	4
1.5.2	b. Vecteurs d'attaque couverts	4
1.5.3	c. Limites des outils existants	4
1.6	Méthodologie de réalisation du projet	4
1.7	Planification du projet	4
1.8	Conclusion	4
2	Conception et architecture du système	5
2.1	Introduction	5
2.2	Principe fondateur : ne jamais lire avant de valider	5
2.3	Étape 1 — Analyse visuelle experte (OpenCV)	5
2.3.1	a. Contrôle de la zone de silence (quiet zone)	5
2.3.2	b. Contrôle d'uniformité de netteté	5
2.3.3	c. Contrôle de structure interne	6
2.4	Étape 2 — Décision de blocage conditionnel	6
2.5	Étape 3 — Lecture sécurisée et extraction d'entités	6
2.6	Résolution DNS complémentaire	6
2.7	Conclusion	6
3	Système de scoring, intégration API et résultats	7
3.1	Introduction	7
3.2	Formulation du problème d'évaluation du risque	7
3.3	Système de scoring — six facteurs heuristiques	7
3.4	Intégration de l'API VirusTotal v3	8
3.5	Implémentation technique	8
3.6	Résultats et études de cas	9
3.6.1	a. Cas 1 — Altération physique détectée (verdict BLOCKED)	9
3.6.2	b. Cas 2 — URL de phishing détectée (verdict HIGH RISK)	9
3.7	Interface graphique et rapport PDF	10
3.8	Discussion des résultats et limites	10
3.9	Conclusion	10
	Conclusion générale & perspectives	11
	Bibliographie	12
	Annexes	13

Chapitre 1 : Contexte général, problématique et étude du besoin

1.1 Introduction

Le Qishing (QR-phishing) est un vecteur d'attaque en forte croissance qui exploite la confiance naturelle des utilisateurs envers les QR Codes pour les rediriger vers des sites malveillants. Contrairement au phishing classique, il ne repose ni sur un e-mail ni sur un lien texte analysable : le contenu malveillant est encapsulé dans une image, ce qui le rend largement invisible aux outils de filtrage traditionnels.

Ce rapport présente la conception et la réalisation d'un outil de sécurité, le QR Code Security Analyzer, capable de détecter à la fois une altération physique du support (autocollant frauduleux apposé sur un QR Code légitime) et une malveillance numérique de la destination encodée (URL de phishing, domaine compromis), avant toute exposition de l'utilisateur au risque.

1.2 Contexte général du projet

Le projet s'inscrit dans le domaine de la cybersécurité appliquée, à l'intersection du traitement d'image (vision par ordinateur) et de l'analyse de réputation de menaces (threat intelligence). Il vise à outiller un utilisateur ou un analyste pour qu'il puisse vérifier, avant de scanner un QR Code affiché dans un lieu public (affiche, menu de restaurant, borne de paiement), qu'il n'a pas été trafiqué physiquement et que la ressource qu'il pointe n'est pas connue comme malveillante.

1.3 Problématique métier et technique

Problématique métier : comment permettre à un utilisateur non technique de vérifier, en quelques secondes, qu'un QR Code rencontré dans l'espace public est sûr à scanner, alors qu'aucun indice visuel évident ne permet de distinguer un code légitime d'un code trafiqué ?

Problématique technique : peut-on concevoir une chaîne de vérification qui bloque la lecture d'un QR Code physiquement suspect avant même sa décodification, puis qui évalue objectivement le risque numérique de sa destination en combinant heuristiques locales et renseignement externe (VirusTotal) ?

1.4 Objectifs du projet

- Détecter l'altération physique (autocollant, falsification) avant toute lecture du contenu du QR Code.
- Extraire et normaliser le contenu décodé : URLs, adresses IP et noms de domaine, de façon structurée.
- Analyser la réputation numérique en croisant une évaluation heuristique locale avec la base VirusTotal (plus de 70 moteurs antivirus).
- Restituer un verdict actionnable : score de risque de 0 à 100, recommandations concrètes et rapport PDF exportable.

1.5 Analyse de la menace : le Qishing

1.5.1 a. Définition et mécanisme d'attaque

Le Qishing consiste à substituer ou superposer un QR Code malveillant à un QR Code légitime (par exemple via un autocollant collé sur une affiche officielle), ou à générer directement un QR Code pointant vers une page de phishing. La victime scanne un code qu'elle croit légitime et est redirigée vers un site frauduleux, souvent conçu pour dérober des identifiants ou des données bancaires.

1.5.2 b. Vecteurs d'attaque couverts

Le projet couvre deux vecteurs d'attaque complémentaires, l'un physique et l'autre numérique :

- **Vecteur physique** : superposition d'un autocollant ou falsification visuelle du support, invisible à l'œil nu dans la majorité des cas.
- **Vecteur numérique** : URL raccourcie, encodée, ou pointant vers un domaine de phishing connu ou récemment enregistré.

1.5.3 c. Limites des outils existants

Les filtres anti-phishing classiques (passerelles de messagerie, extensions de navigateur) n'inspectent pas le contenu visuel d'un QR Code : ils n'interviennent qu'après le clic sur un lien, une fois l'utilisateur déjà engagé dans le processus de navigation. Une attaque physique est, elle, totalement absente de leur périmètre de détection, ce qui justifie une approche dédiée intervenant en amont, au moment même de l'acquisition de l'image.

1.6 Méthodologie de réalisation du projet

Le projet applique une méthode stricte en trois étapes séquentielles et non contournables : analyse visuelle experte, décision de blocage conditionnel, puis lecture sécurisée uniquement si les deux premières étapes valident l'intégrité physique du support. Ce séquençement garantit qu'une URL ne soit jamais exposée à l'utilisateur ou soumise aux services externes tant que la légitimité physique du QR Code n'est pas établie.

1.7 Planification du projet

Phase	Activités principales	Livrable
Cadrage & menace	Étude du Qishing, définition des vecteurs d'attaque et des objectifs	Chapitre 1
Conception	Architecture en 3 étapes, contrôles visuels OpenCV	Chapitre 2
Scoring & API	Heuristiques de risque, intégration VirusTotal, interface	Chapitre 3
Industrialisation	Interface CustomTkinter, rapport PDF, historique JSON	Application + rapport

1.8 Conclusion

Ce premier chapitre a posé le cadre du projet et détaillé la menace du Qishing, dont les caractéristiques — absence de lien texte, dimension physique de l'attaque — justifient une approche de vérification dédiée. Le chapitre suivant détaille la conception et l'architecture du système de vérification en trois étapes.

Chapitre 2 : Conception et architecture du système

2.1 Introduction

Ce chapitre décrit l'architecture logicielle retenue pour répondre à la problématique posée : une méthode stricte en trois étapes, dans laquelle le contenu du QR Code n'est jamais décodé tant que son intégrité physique n'a pas été validée par une analyse d'image experte.

2.2 Principe fondateur : ne jamais lire avant de valider

Une URL peut être numériquement « propre » tout en servant un QR Code physiquement trafiqué : un autocollant malveillant peut pointer vers une adresse qui n'a pas encore été signalée par les bases de réputation. L'analyse visuelle intervient donc en amont du décodage, et non en complément, ce qui inverse l'ordre habituel des systèmes d'analyse d'URL.



Le contenu n'est JAMAIS decode tant que l'integrite physique n'est pas validee

Figure 2.1: Architecture en 3 étapes strictes du QR Code Security Analyzer.

2.3 Étape 1 — Analyse visuelle experte (OpenCV)

Trois contrôles indépendants sont exécutés avant toute tentative de décodage, sur l'image acquise (fichier ou flux webcam) :

2.3.1 a. Contrôle de la zone de silence (quiet zone)

Le contour du QR Code est détecté par seuillage de Canny puis approximation polygonale. Le masque obtenu est dilaté de 15 pixels afin d'isoler l'anneau périphérique correspondant à la zone de silence normalement blanche. Si l'intensité moyenne de cet anneau descend sous 200/255, la zone de silence est jugée violée ou débordante — signe d'une superposition.

2.3.2 b. Contrôle d'uniformité de netteté

La variance du Laplacien, mesure standard de netteté, est calculée séparément sur la région centrale du code et sur sa périphérie. Un ratio de netteté inférieur à 0,6 entre les deux zones révèle une rupture de texture, signature typique d'un autocollant apposé sur une surface différente de celle du support d'origine.

2.3.3 c. Contrôle de structure interne

L'écart-type des niveaux de gris à l'intérieur du contour est mesuré, complété par un comptage des motifs carrés détectés par analyse de contours internes. Un écart-type inférieur à 30, ou moins de 15 carrés internes détectés, indique une structure interne trop uniforme ou trop dégradée pour être un module QR Code valide.

Si l'un de ces trois contrôles échoue, l'image est classée SUSPICIOUS ou COVERED/DAMAGED et la lecture Pyzbar n'est jamais exécutée.

2.4 Étape 2 — Décision de blocage conditionnel

Cette étape agit comme un verrou binaire : dès qu'un statut visuel autre que LEGITIMATE est renvoyé par l'étape 1, l'analyse est immédiatement interrompue et un verdict BLOCKED est retourné avec un score de risque forcé à 100/100. Le contenu du QR Code n'est jamais révélé à l'utilisateur, ce qui élimine toute exposition, même passive, à une destination potentiellement malveillante.

2.5 Étape 3 — Lecture sécurisée et extraction d'entités

Une fois l'intégrité physique validée, la bibliothèque Pyzbar décode le contenu du QR Code. Trois familles d'entités sont ensuite extraites par expressions régulières, dédoublées puis structurées :

- **URLs** : motif `http(s)://` complet, extrait et dédoublé.
- **Adresses IP** : IPv4 validées, chaque octet devant être compris entre 0 et 255.
- **Domaines** : extraits des URLs identifiées, complétés par un motif générique de nom de domaine appliqué au texte brut.

2.6 Résolution DNS complémentaire

Chaque domaine extrait est résolu via `socket.getaddrinfo()` afin d'obtenir ses adresses IP réelles. Ces IPs résolues enrichissent la liste soumise à l'analyse de réputation, ce qui est utile lorsque le nom de domaine seul ne suffit pas à révéler l'infrastructure d'hébergement sous-jacente.

Par mesure de sécurité pour l'analyste, toutes les URLs, IPs et domaines sont systématiquement « défangés » (`hxxp://`, `[.]`) avant tout affichage à l'écran ou export dans le rapport PDF, ce qui rend impossible un clic accidentel sur un lien malveillant depuis l'interface.

2.7 Conclusion

L'architecture en trois étapes garantit qu'aucune donnée potentiellement dangereuse n'est exposée avant qu'un premier filtre visuel n'ait validé l'intégrité physique du support. Le chapitre suivant détaille le système de scoring appliqué au contenu une fois celui-ci lu, ainsi que son intégration avec le service externe VirusTotal.

Chapitre 3 : Système de scoring, intégration API et résultats

3.1 Introduction

Ce chapitre présente la phase d'évaluation du risque numérique : formulation du problème de scoring, description des six facteurs heuristiques retenus, intégration de l'API VirusTotal v3, puis analyse de deux études de cas représentatives issues de l'application.

3.2 Formulation du problème d'évaluation du risque

Une fois le contenu du QR Code lu en toute sécurité, le système doit produire une évaluation objective et reproductible du risque associé à l'URL extraite. Le choix retenu est un score heuristique cumulatif, borné à 100 points, combinant des règles locales déterministes et, lorsque disponible, une consultation de la base de réputation VirusTotal. Cette approche par score explicite, plutôt qu'un modèle de classification opaque, a été privilégiée pour sa transparence : chaque point de risque est justifié par une raison lisible, restituée à l'utilisateur.

3.3 Système de scoring — six facteurs heuristiques

La classe `RiskScorer` évalue six facteurs indépendants, chacun contribuant un nombre de points plafonné, la somme totale étant elle-même plafonnée à 100 :

Facteur	Poids max.	Règle de détection
HTTPS absent	15 pts	L'URL ne commence pas par <code>https://</code>
IP brute dans l'URL	20 pts	Une adresse IPv4 est utilisée à la place d'un nom de domaine
Raccourcisseur d'URL	15 pts	Domaine appartenant à une liste de services connus (bit.ly, tinyurl.com, t.co, ...)
Mots-clés suspects	20 pts	5 points par mot-clé détecté (login, verify, secure, bank, ...), plafonné à 20
Longueur de l'URL	10 pts	10 pts si > 200 caractères, 5 pts si > 100 caractères
Réputation VirusTotal	20 pts	5 points par moteur signalant l'entité comme malveillante, plafonné à 20

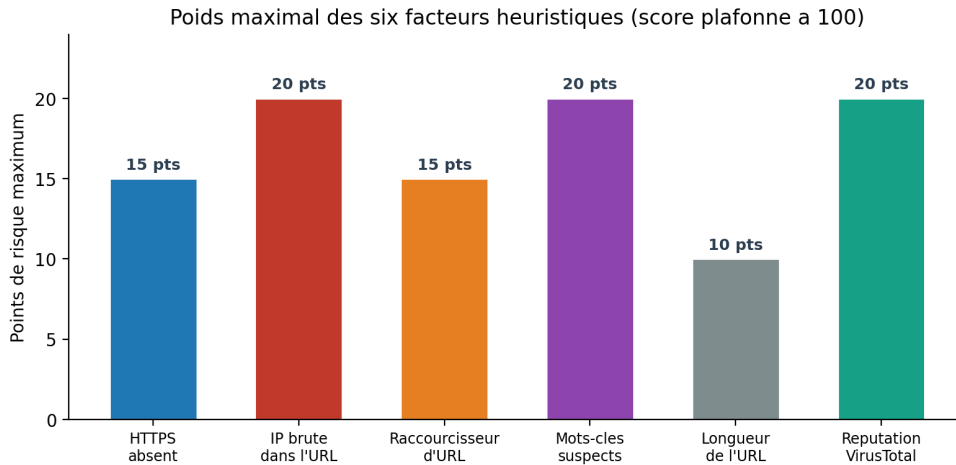


Figure 3.1: Poids maximal de chacun des six facteurs heuristiques du score de risque.

Le score cumulé détermine ensuite un verdict qualitatif selon cinq bandes de sévérité croissante :



Figure 3.2: Échelle de verdict associée au score de risque (*get_verdict*).

3.4 Intégration de l'API VirusTotal v3

Trois points de terminaison de l'API VirusTotal v3 sont interrogés séquentiellement pour chaque analyse, avec une pause d'une seconde entre chaque appel afin de respecter le quota du plan public :

- GET /urls/{url_id} — l'URL est encodée en base64 URL-safe sans padding avant l'appel.
- GET /ip_addresses/{ip} — pour chaque IP extraite directement ou résolue par DNS.
- GET /domains/{domain} — pour chaque nom de domaine détecté.

Les réponses sont traitées de façon robuste selon le code retourné : un code 200 permet d'extraire les statistiques `last_analysis_stats` (compteurs *malicious* et *suspicious*) ; un code 404 signale une entité inconnue de VirusTotal (`NOT_FOUND_IN_VT`) ; un code 429 indique un quota dépassé (`RATE_LIMIT_EXCEEDED`) et est géré sans provoquer d'interruption de l'application.

3.5 Implémentation technique

L'application est développée en Python. Le traitement d'image et la détection de contours reposent sur OpenCV, le décodage sur Pyzbar, l'interface graphique sur CustomTkinter et la génération de rapports sur ReportLab. La persistance de l'historique des analyses est assurée par un fichier JSON local (`history.json`), et les appels réseau (résolution DNS, requêtes VirusTotal) utilisent respectivement le module `socket` et la bibliothèque `requests`.

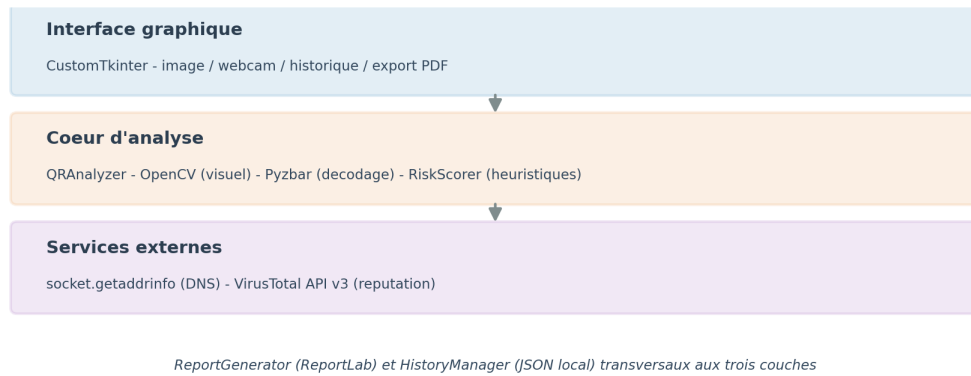


Figure 3.3: Architecture logicielle en trois couches de l'application.

3.6 Résultats et études de cas

3.6.1 a. Cas 1 — Altération physique détectée (verdict **BLOCKED**)

Dans ce scénario, un autocollant a été apposé sur un QR Code authentique. Le contrôle d'uniformité de netteté mesure un ratio de 0,42 entre le centre et la périphérie du code, très inférieur au seuil de blocage de 0,60. Le verdict **BLOCKED** est retourné avant toute exécution de Pyzbar, le score de risque est forcé à 100/100 sans ambiguïté possible, et le contenu du QR Code n'est jamais révélé à l'utilisateur. La recommandation associée invite à signaler l'incident à la sécurité physique du site.

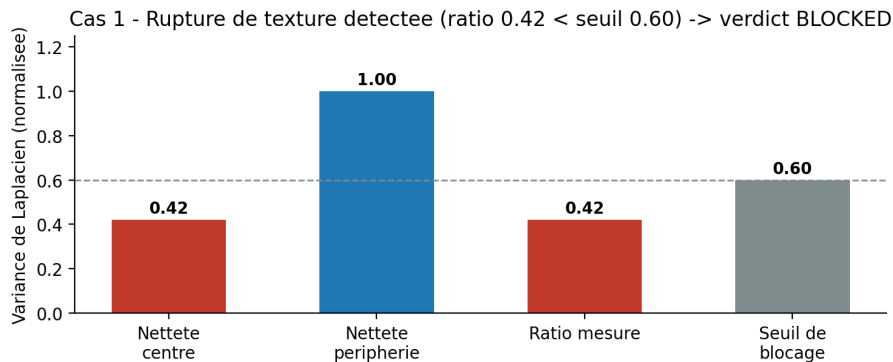


Figure 3.4: Cas 1 : rupture de netteté mesurée entre centre et périphérie ($0,42 < \text{seuil } 0,60$).

Ce scénario illustre l'intérêt de bloquer avant de lire : décoder puis analyser l'URL exposerait déjà l'utilisateur au risque que l'on cherche précisément à prévenir.

3.6.2 b. Cas 2 — URL de phishing détectée (verdict **HIGH RISK**)

Dans ce second scénario, le QR Code est visuellement sain (les trois contrôles de l'étape 1 passent) mais numériquement malveillant. L'URL décodée est `hxxp://secure-paypa1-verify-account[.]tk/login/conn` (défangée dans ce rapport). Le score cumulé atteint 55/100, décomposé comme suit : absence de HTTPS (+15), six mots-clés suspects détectés — login, verify, secure notamment — (+20), et neuf moteurs VirusTotal signalant l'entité comme malveillante (+20). Le verdict final est **HIGH RISK**.

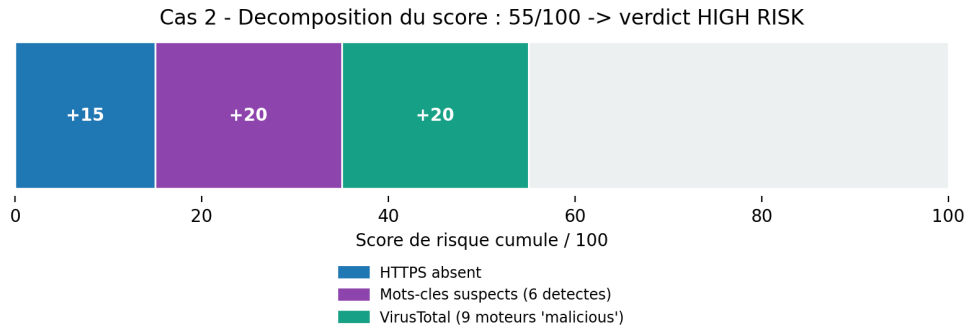


Figure 3.5: Cas 2 : décomposition du score de risque cumulé (55/100).

3.7 Interface graphique et rapport PDF

L'interface graphique, développée avec CustomTkinter, propose une barre latérale (analyse d'image, webcam, historique, effacement) et une zone principale affichant l'aperçu de l'image, le verdict, le score et trois onglets : journal d'analyse, facteurs de risque et données extraites. Un bouton dédié génère un rapport PDF via `ReportGenerator`, reprenant le verdict, le tableau détaillé des facteurs de risque, les entités défangées et des recommandations contextuelles adaptées à chaque facteur déclenché (`get_recommendations`). Chaque analyse est par ailleurs journalisée dans un historique JSON local consultable depuis l'interface.

3.8 Discussion des résultats et limites

Le séquençement strict de l'architecture — blocage avant lecture — constitue la principale garantie de sécurité du système : même en l'absence de clé API VirusTotal ou en cas de quota dépassé, la détection d'altération physique reste pleinement opérationnelle et suffit à elle seule à intercepter les attaques de type Qishing physique.

Une limite technique a été observée en test : les émojis Unicode utilisés dans les recommandations ne s'affichent pas nativement dans ReportLab avec la police Helvetica et sont remplacés par un caractère de substitution dans le PDF exporté. Une correction simple consiste à intégrer une police supportant les emoji ou à leur substituer un texte simple. Par ailleurs, les seuils utilisés pour les contrôles visuels (ratio de netteté à 0,6, intensité de zone de silence à 200/255) sont fixes et calibrés empiriquement ; ils gagneraient à être validés statistiquement sur un corpus plus large d'images réelles.

3.9 Conclusion

La phase de scoring et d'intégration API a permis de combiner trois couches de défense indépendantes — analyse visuelle, heuristiques locales et réputation externe — pour produire un verdict actionnable. Les deux études de cas illustrent la capacité du système à intercepter aussi bien une attaque physique qu'une attaque purement numérique, avec un raisonnement transparent et traçable à chaque étape.

Conclusion générale & perspectives

Ce projet a mis en œuvre une chaîne complète de vérification de sécurité pour QR Codes, depuis l'acquisition de l'image jusqu'à un rapport PDF exportable, en passant par une interface graphique fonctionnelle. La démarche adoptée — ne jamais lire avant de valider — structure l'ensemble du système autour d'un principe de sécurité par conception, rarement appliqué avec cette rigueur dans les outils grand public existants.

Les résultats obtenus sur les scénarios testés confirment la pertinence de la méthode en trois étapes : une altération physique est interceptée avant toute exposition au risque numérique, tandis qu'une URL de phishing propre visuellement mais malveillante numériquement est correctement classée grâce à la combinaison d'heuristiques locales et de renseignement externe.

Plusieurs perspectives d'amélioration peuvent être envisagées :

- Entraîner un classifieur supervisé sur des exemples réels d'autocollants, afin de dépasser les seuils fixes actuels (ratio de netteté, intensité de zone de silence).
- Étendre la liste de mots-clés suspects par apprentissage plutôt que par une liste statique codée en dur.
- Ajouter une file d'attente asynchrone pour respecter le quota VirusTotal à grande échelle, sans bloquer l'interface utilisateur.
- Publier un tableau de bord web de l'historique des analyses, en remplacement du fichier JSON local.

Ce projet démontre que la sécurité d'un QR Code ne se limite pas à son contenu numérique, que le traitement d'image classique reste pertinent face à des menaces modernes, et qu'un verdict de sécurité doit être actionnable pour l'utilisateur final, et non seulement informatif.

Il convient de rappeler que l'outil développé est un projet académique de fin d'études et ne saurait se substituer à un dispositif de sécurité physique ou à un outil de threat intelligence certifié en environnement de production.

Bibliographie

1. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
2. VirusTotal. *VirusTotal API v3 Documentation*. <https://docs.virustotal.com/reference/overview>
3. Kirstein, J. et al. *pyzbar — Python library for reading barcodes and QR codes with zbar*. PyPI.
4. Watson, T. et al. *reportlab-lib — PDF generation toolkit for Python*. ReportLab.com.
5. Tom Schimansky. *CustomTkinter — Modern UI-library for Python based on Tkinter*. GitHub.
6. ISO/IEC 18004:2015. *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*.
7. OWASP Foundation. *Phishing and QR-code-based attack vectors* — Community references.

Annexes

Annexe A — Architecture du pipeline complet

Le pipeline s'articule en sept étapes : (1) acquisition de l'image (fichier ou flux webcam), (2) analyse visuelle experte (zone de silence, netteté, structure interne), (3) décision de blocage conditionnel, (4) lecture Pyzbar sécurisée, (5) extraction et normalisation des entités par expressions régulières, (6) résolution DNS complémentaire et interrogation de l'API VirusTotal, (7) calcul du score de risque, restitution du verdict et génération optionnelle d'un rapport PDF.

Annexe B — Stack technique complète

Composant	Rôle
OpenCV (cv2)	Détection de contours, calcul de netteté (Laplacien), analyse de zone de silence
Pyzbar	Décodage des QR Codes une fois l'intégrité physique validée
CustomTkinter	Interface graphique de bureau (thème sombre)
ReportLab	Génération du rapport PDF (SimpleDocTemplate, Table, Paragraph)
requests	Appels à l'API REST VirusTotal v3
socket	Résolution DNS complémentaire des domaines extraits
python-dotenv	Chargement de la clé API VirusTotal depuis un fichier .env

Annexe C — Structure d'une entrée d'historique (JSON)

Chaque analyse est journalisée dans `history.json` avec les champs suivants : `timestamp` (horodatage ISO), `verdict` (SAFE à CRITICAL, ou BLOCKED / UNREADABLE), `risk_score` (0-100), `risk_factors` (détail par facteur), `raw_data` (contenu décodé, ou message de blocage), et `entities` (listes d'URLs, d'IPs et de domaines extraits).